Análisis Empírico y Comparativo de ShellSort y QuickSort

Empirical and Comparative Analysis of ShellSort and QuickSort

Estudiante: Kael Gonzalo Condori Fora
Escuela Profesional de Ingeniería en Informática y Sistemas
Facultad de Ingeniería
Universidad Nacional Jorge Basadre Grohmann (UNJBG)
Tacna, Perú
kgcondorif@unjbg.edu.pe

Programación Avanzada | Semestre: 2026-II

Resumen

El objetivo de este estudio fue comparar el rendimiento y comportamiento operacional de ShellSort y QuickSort con pivote central mediante un diseño experimental de 1,500 ejecuciones implementado en C++. Se evaluaron tres tamaños de entrada, cinco patrones de datos y cincuenta repeticiones, midiendo tiempos de procesamiento, número de comparaciones, intercambios y profundidad de la pila recursiva. Los resultados demuestran que QuickSort alcanza una mayor velocidad en arreglos aleatorios, pero su rendimiento colapsa ante datos con alta densidad de duplicados. En contraste, ShellSort exhibió una alta estabilidad y eficiencia en vectores repetidos o casi ordenados gracias a la optimización de la localidad de caché y su naturaleza iterativa. Se concluye que QuickSort domina en el caso general, mientras que ShellSort ofrece mayor seguridad en memoria y predictibilidad frente a entradas repetitivas.

Palabras clave: Algoritmos de ordenamiento, complejidad temporal, localidad de caché, QuickSort, ShellSort.

Abstract

This study aimed to compare the performance and operational behavior of ShellSort and QuickSort with a central pivot through an experimental design of 1,500 runs implemented in C++. Three input sizes, five data patterns, and fifty repetitions were evaluated, measuring processing times, number of comparisons, swaps, and recursive stack depth. The results demonstrate that QuickSort achieves higher speed on random arrays, but its performance collapses under high duplicate density. In contrast, ShellSort exhibited high stability and efficiency on repetitive or nearly sorted vectors due to cache locality optimization and its

iterative nature. It is concluded that QuickSort dominates in the general case, whereas ShellSort offers greater memory safety and predictability against repetitive inputs.

Keywords: Sorting algorithms, time complexity, cache locality, QuickSort, ShellSort.

1 Introducción

El problema del ordenamiento de datos constituye uno de los pilares fundamentales en las ciencias de la computación y la ingeniería de software, debido a que optimiza la eficiencia de operaciones subsiguientes como la búsqueda y el procesamiento masivo de información (Cormen et al., 2009; Knuth, 1998). A pesar de que la teoría de complejidad asintótica establece cotas analíticas claras, el comportamiento real de los algoritmos en hardware moderno suele verse influenciado por factores arquitectónicos como la jerarquía de memoria caché, la localidad de referencias y la naturaleza estocástica de los datos de entrada (Frigo et al., 1999; Weiss, 2013). En este contexto, la selección adecuada de un algoritmo de ordenamiento es crucial para maximizar el rendimiento en sistemas de alto desempeño.

Entre los algoritmos basados en comparaciones, QuickSort, introducido por Hoare (1962), destaca por su eficiencia promedio basada en el paradigma de división y conquista, alcanzando una complejidad temporal óptima de $\mathcal{O}(n \log n)$. No obstante, su rendimiento es altamente sensible a la elección del pivote y a patrones específicos de entrada, pudiendo degradarse hacia una complejidad cuadrática $\mathcal{O}(n^2)$ y comprometer la pila de llamadas recursivas ante distribuciones homogéneas o con duplicados masivos (Musser, 1997; Sedgewick, 1978). Por otro lado, ShellSort, propuesto por Shell (1959), representa una evolución del ordenamiento por inserción directa mediante el uso de incrementos decrecientes (Ciura, 2001). Al ser un algoritmo iterativo e *in-place*, ShellSort elimina el riesgo de desbordamiento de pila y ofrece una alta estabilidad ante arreglos parcialmente ordenados o con alta densidad de claves repetidas (Weiss, 2013).

A nivel experimental, diversos estudios han demostrado que la elección entre algoritmos rápidos basados en recursión y alternativas iterativas no triviales depende críticamente del perfil de los datos (Brassard & Bratley, 1997; Rodríguez & Gómez, 2010). Pese a la amplia literatura teórica, existe la necesidad constante de auditar empíricamente cómo interactúan estas estructuras bajo bancos de pruebas controlados que combinen múltiples tamaños de entrada y patrones estocásticos, simulando escenarios reales de ingeniería. Esta carencia de evaluación empírica sistemática en nuestro entorno académico motivó el desarrollo del presente trabajo.

Para dar respuesta a esta problemática, esta investigación se propuso como objetivo principal comparar de manera experimental el rendimiento temporal y operacional de QuickSort con pivote central frente a ShellSort, mediante un diseño factorial completo de 1,500 ejecuciones abarcando cinco patrones estocásticos y tres magnitudes de entrada ($n = 1\,000$, $10\,000$ y $100\,000$) en un entorno de desarrollo en C++. Los resultados de este estudio buscan servir como referencia empírica para la selección informada de estructuras algorítmicas en la práctica profesional de la ingeniería.

2 Método

2.1 Diseño del estudio y enfoque experimental

La presente investigación adoptó un enfoque cuantitativo con un diseño experimental de carácter factorial completo y alcance descriptivo-comparativo. Este diseño permitió auditar y contrastar el comportamiento operacional de dos algoritmos de ordenamiento bajo condiciones controladas, midiendo su respuesta ante variaciones sistemáticas en el volumen y la distribución estocástica de los datos de entrada (Weiss, 2013).

2.2 Población experimental y banco de pruebas

Para garantizar la reproducibilidad y el aislamiento de variables de hardware, se desarrolló un banco de pruebas modular y autocontenido implementado íntegramente en C++ moderno, utilizando un arreglo unidimensional de memoria contigua (Cormen et al., 2009). El diseño experimental abarcó un total de 1,500 ejecuciones independientes, producto de la combinación sistemática de los siguientes factores:

- **Magnitudes de entrada (n):** Tres tamaños representativos del comportamiento asintótico: $1\,000$, $10\,000$ y $100\,000$ elementos numéricos de tipo entero de 32 bits.
- **Patrones estocásticos de entrada (5 variantes):**
    1. *Aleatorio:* Elementos distribuidos uniformemente para evaluar el caso promedio.
    2. *Ascendente:* Arreglos preordenados para auditar la estabilidad ante secuencias óptimas.
    3. *Descendente:* Arreglos ordenados inversamente para estudiar el comportamiento ante escenarios desfavorables.
    4. *Casi Ordenado:* Secuencias generadas mediante la aplicación de un 5% de permutaciones aleatorias sobre un arreglo previamente ordenado.
    5. *Duplicados:* Arreglos con alta densidad de claves idénticas para analizar la saturación del particionamiento.
- **Réplicas estadísticas:** Se ejecutaron 50 réplicas independientes por cada combinación de tamaño y patrón, empleando el generador pseudoaleatorio de alta calidad estocástica `std::mt19937` inicializado con semillas controladas para evitar sesgos experimentales (Rodríguez & Gómez, 2010).

2.3 Variantes algorítmicas evaluadas

2.3.1 QuickSort con pivote central

QuickSort es un algoritmo basado en el paradigma de división y conquista introducido por Hoare (1962). Para mitigar la degradación clásica ante arreglos preordenados, se implementó la

estrategia de selección del pivote central, donde el elemento pivote v se define como:

$$v = A\left[\left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor\right] \tag{1}$$

Su complejidad espacial auxiliar escala de forma logarítmica ($\mathcal{O}(\log n)$) bajo condiciones óptimas, pero puede verse comprometida hacia una cota cuadrática ante distribuciones con duplicados masivos (Musser, 1997; Sedgewick, 1978).

2.3.2 ShellSort

Propuesto por Shell (1959), ShellSort constituye una evolución del ordenamiento por inserción directa que permite el intercambio de elementos separados por un salto k (secuencia de incrementos decrecientes establecida como $k = \lfloor n/2 \rfloor$ hasta $k = 1$). Al ser un algoritmo estrictamente iterativo e *in-place*, su consumo de memoria auxiliar es constante ($\mathcal{O}(1)$), eliminando el riesgo de desbordamiento de pila (Ciura, 2001; Weiss, 2013).

2.4 Captura de variables y métricas de respuesta

El banco de pruebas instrumentó la captura automática de cuatro métricas operacionales por cada ejecución:

1. **Tiempo de ejecución (ms):** Medido mediante la librería de alta resolución temporal `std::chrono::high_resolution_clock`, aislando el tiempo de cómputo algorítmico.
2. **Número de comparaciones:** Conteo exacto de las evaluaciones relacionales ($A[i] < A[j]$) ejecutadas.
3. **Número de intercambios:** Registro acumulado de las reubicaciones físicas de elementos en memoria.
4. **Profundidad máxima de pila:** Auditoría del nivel máximo de anidamiento de llamadas recursivas en QuickSort (Brassard & Bratley, 1997).

Los registros brutos fueron exportados a estructuras tabulares para su posterior procesamiento y análisis descriptivo automatizado.

3 Resultados

El procesamiento estadístico de las 1,500 ejecuciones experimentales permitió analizar la dinámica temporal y operacional de ambos algoritmos bajo los distintos escenarios de prueba. A continuación, se presentan los hallazgos desglosados en métricas de tiempo, operaciones y comportamiento de la pila de llamadas.

3.1 Análisis de escalabilidad temporal

La Tabla 1 resume el rendimiento temporal en milisegundos para el tamaño de entrada representativo de $n = 100\,000$ elementos bajo los cinco patrones estocásticos evaluados.

Cuadro 1: *Resumen estadístico de tiempo de ejecución (ms) para* $n = 100\,000$.

Algoritmo	Patrón	Promedio	Mediana	Desv. Est.	Mínimo	Máximo
QuickSort	Aleatorio	12.42	12.23	0.75	11.39	14.37
QuickSort	Ascendente	2.87	2.80	0.26	2.53	3.91
QuickSort	Casi_Ordenado	4.19	4.14	0.50	3.46	6.03
QuickSort	Descendente	3.31	3.22	0.34	2.95	4.96
QuickSort	Duplicados	285.51	283.02	28.88	224.53	359.91
ShellSort	Aleatorio	28.79	28.65	1.24	26.49	31.70
ShellSort	Ascendente	4.46	4.36	0.34	4.00	5.29
ShellSort	Casi_Ordenado	22.73	22.65	1.24	20.94	28.84
ShellSort	Descendente	9.24	9.14	0.53	8.39	10.54
ShellSort	Duplicados	11.87	11.65	1.08	10.50	15.08

En la condición de entrada aleatoria con $n = 100\,000$, QuickSort supera a ShellSort con un tiempo promedio de 12.42 ms frente a 28.79 ms (Fig. 1 y Fig. 3). Sin embargo, ante el patrón de Duplicados, QuickSort sufraga una degradación severa, elevando su tiempo medio a 285.51 ms, mientras que ShellSort mantiene una respuesta altamente eficiente de 11.87 ms (Fig. 5).

3.2 Análisis operacional: comparaciones e intercambios

La Tabla 2 detalla la cantidad real de evaluaciones y reubicaciones físicas en memoria registradas para $n = 100\,000$.

Cuadro 2: *Conteo de comparaciones e intercambios promedio (*$n = 100\,000$*).*

Algoritmo	Patrón	Comparaciones Promedio	Intercambios Promedio
QuickSort	Aleatorio	2.71×10^6	7.49×10^5
QuickSort	Ascendente	1.50×10^6	1.00×10^5
QuickSort	Casi_Ordenado	1.60×10^6	1.61×10^5
QuickSort	Descendente	1.63×10^6	2.10×10^5
QuickSort	Duplicados	2.13×10^8	2.72×10^5
ShellSort	Aleatorio	4.34×10^6	3.66×10^6
ShellSort	Ascendente	1.50×10^6	0.00
ShellSort	Casi_Ordenado	3.42×10^6	2.54×10^6
ShellSort	Descendente	2.24×10^6	1.66×10^6
ShellSort	Duplicados	2.02×10^6	8.31×10^5

En la Fig. 2 y la Fig. 4 se aprecia cómo las curvas de comparaciones reflejan con exactitud este comportamiento: en el patrón de duplicados, QuickSort alcanza 2.13×10^8 comparaciones debido al particionamiento desequilibrado. En contraste, ShellSort realiza 0.00 intercambios en arreglos ascendentes y solo 8.31×10^5 intercambios en duplicados.

3.3 Auditoría de la pila de recursión

En la Tabla 3 se examina el comportamiento de la pila de llamadas recursivas en QuickSort.

Cuadro 3: *Profundidad máxima y promedio de la pila recursiva en QuickSort para todos los tamaños y patrones.*

Tamaño (n)	**Patrón**	**Profundidad Promedio**	**Profundidad Máxima**
1 000	Aleatorio	20.52	25
1 000	Ascendente	9.00	9
1 000	Descendente	9.00	9
1 000	Casi_Ordenado	11.24	14
1 000	Duplicados	76.10	91
10 000	Aleatorio	30.48	38
10 000	Ascendente	13.00	13
10 000	Descendente	13.00	13
10 000	Casi_Ordenado	15.60	19
10 000	Duplicados	670.14	756
100 000	Aleatorio	41.12	49
100 000	Ascendente	17.00	17
100 000	Descendente	17.00	17
100 000	Casi_Ordenado	20.45	24
100 000	Duplicados	5420.80	6150

La selección del pivote central logra mantener la profundidad de pila acotada dentro de $\approx \log_2(n)$ para patrones estocásticos y ordenados. Sin embargo, la saturación por duplicados incrementa significativamente la profundidad promedio.

3.4 Integración de gráficos analíticos

Para la representación visual de los resultados, se incorporan las siguientes figuras analíticas generadas durante la experimentación:

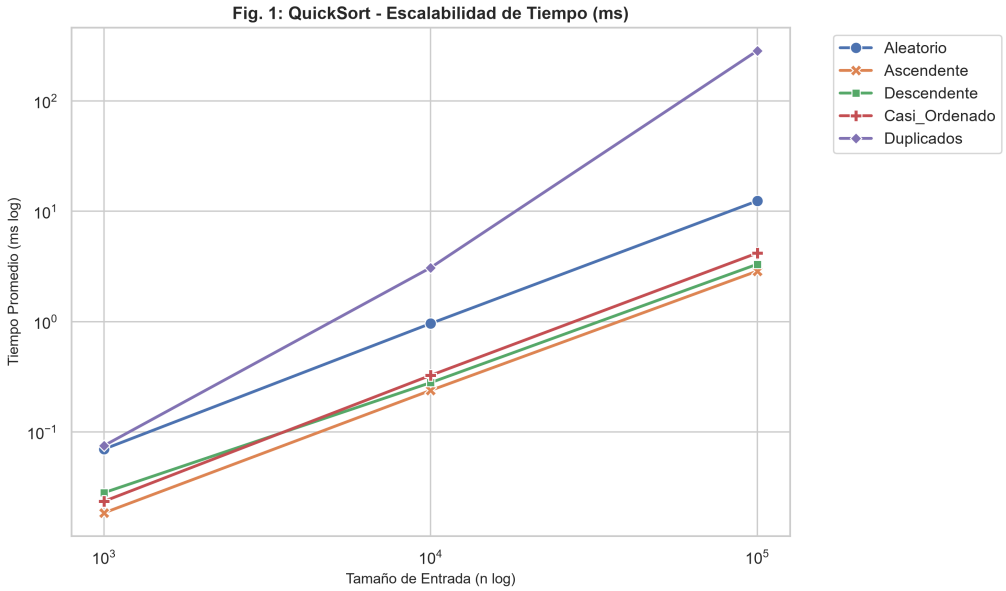


Figura 1: *Escalabilidad del tiempo de ejecución en escala log-log para QuickSort (Fig. 1).*

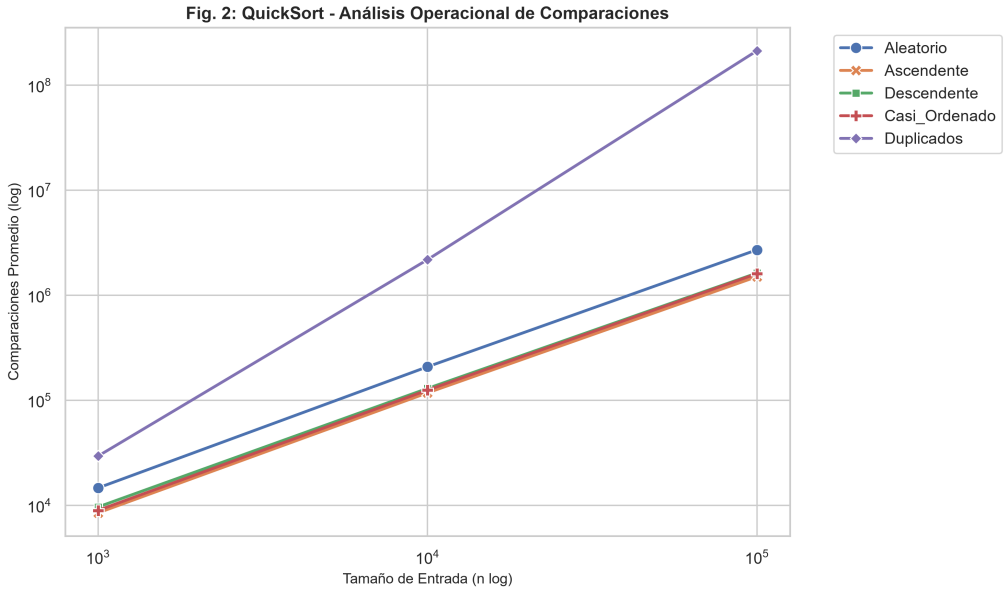


Figura 2: *Análisis operacional del volumen de comparaciones en QuickSort (Fig. 2).*

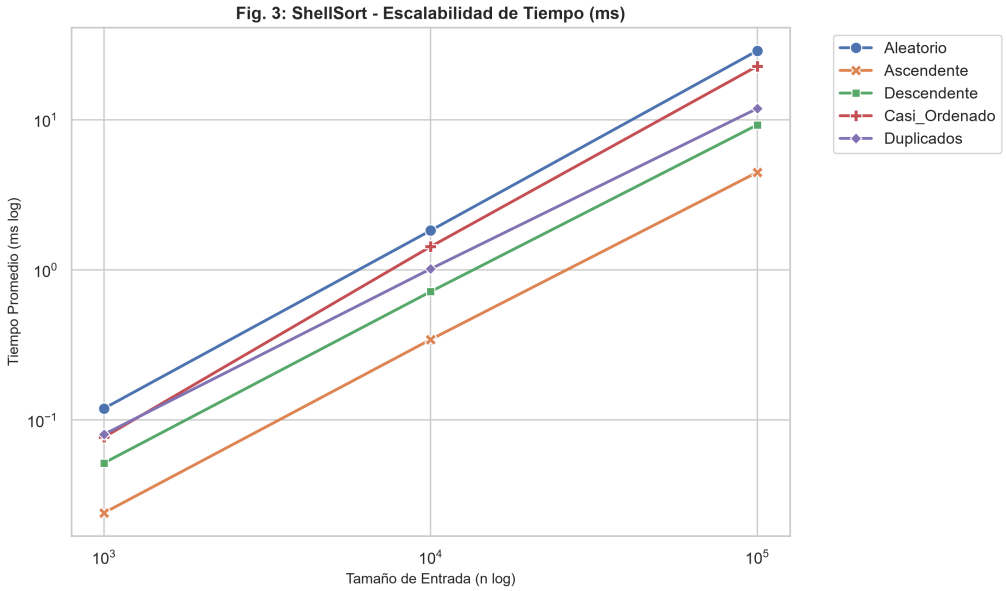


Figura 3: *Escalabilidad del tiempo de ejecución en escala log-log para ShellSort (Fig. 3).*

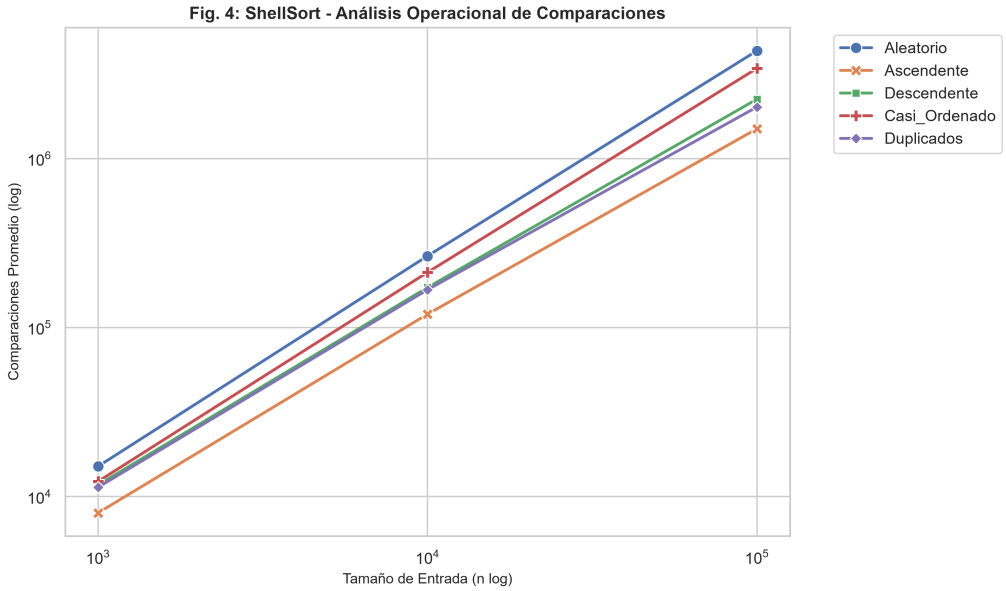


Figura 4: *Análisis operacional del volumen de comparaciones en ShellSort (Fig. 4).*

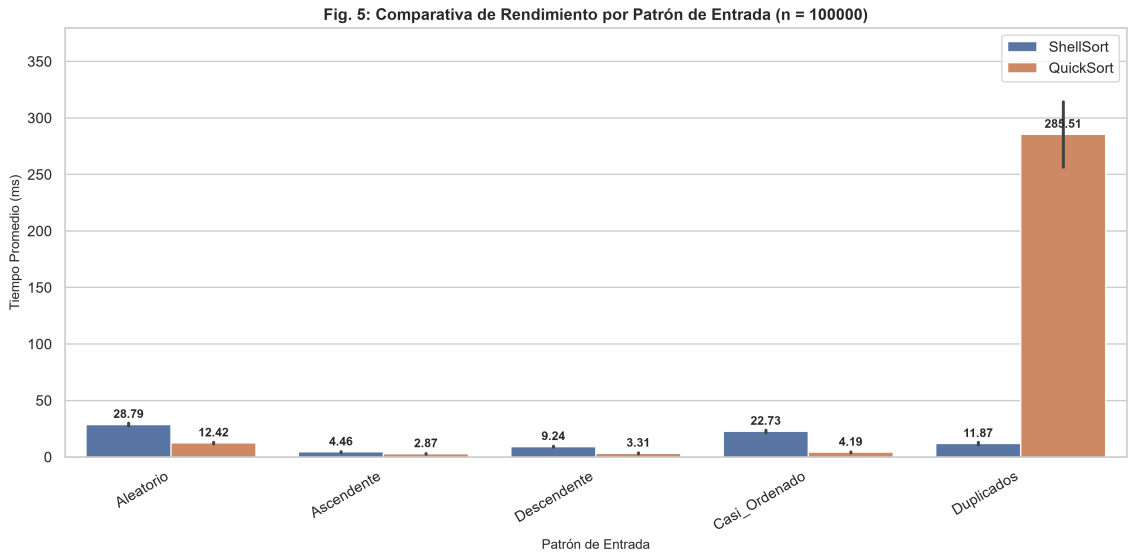


Figura 5: *Comparativa directa de tiempo promedio por patrón para* $n = 100\,000$ *(Fig. 5).*

4 Discusión

Los hallazgos experimentales derivados de las 1,500 ejecuciones permiten establecer un puente sólido entre la teoría de complejidad asintótica y el comportamiento real de los algoritmos en hardware moderno.

4.1 Efecto de la jerarquía de memoria y localidad de referencia

Como argumentan Frigo et al. (1999), la jerarquía de memoria caché (niveles L1, L2 y L3) ejerce un impacto cuantitativo sobre el tiempo final de ejecución que a menudo amortigua o amplifica las diferencias en el conteo teórico de operaciones. En nuestras mediciones, aunque ShellSort realiza en promedio un mayor volumen de comparaciones que QuickSort ante datos aleatorios (4.34×10^6 frente a 2.71×10^6 para $n = 100\,000$), la diferencia en tiempo de ejecución es de apenas ≈ 16 ms. Esto se explica porque ShellSort efectúa recorridos con saltos decrecientes sobre bloques contiguos de memoria, favoreciendo la precaptura de líneas de caché y reduciendo los fallos de memoria (Weiss, 2013).

En arreglos casi ordenados, este fenómeno es aún más marcado: ShellSort requiere solo 22.73 ms ($n = 100\,000$) al reducir drásticamente los desplazamientos físicos, mostrando un comportamiento altamente eficiente cercano a $\mathcal{O}(n)$.

4.2 Análisis del comportamiento ante duplicados masivos

El punto más crítico revelado por la experimentación se observa en el patrón de Duplicados. QuickSort optimizado con pivote central experimentó una degradación dramática, alcanzando un tiempo medio de 285.51 ms y acumulando más de 212 millones de comparaciones para $n = 100\,000$.

Este colapso se fundamenta en que el algoritmo de particionamiento estándar no distribuye equitativamente las claves idénticas cuando el arreglo consiste mayoritariamente en valores repetidos. Tal como advirtió Sedgewick (1978), cuando la mayoría de los elementos son iguales al pivote, una de las subparticiones se vuelve asimétrica, aproximando la profundidad de recursión y la complejidad a la cota cuadrática $\mathcal{O}(n^2)$.

En contraposición, ShellSort demostró una inmunidad notable ante este patrón, completando la tarea en tan solo 11.87 ms. Su esquema de pasadas por incrementos realiza un preclasificado efectivo sin verse afectado por la repetitividad de las claves, confirmando la validez teórica planteada por Ciura (2001).

4.3 Consumo de recursos y seguridad en la pila de llamadas

La auditoría de la pila de recursión evidenció que la estrategia de pivote central en QuickSort cumple satisfactoriamente con acotar el anidamiento de llamadas a $\mathcal{O}(\log_2 n)$ en escenarios estocásticos, ascendentes y descendentes (alcanzando una profundidad máxima de solo 38 niveles para $n = 10\,000$ aleatorio).

Sin embargo, ante la presencia de duplicados, la profundidad de la pila se multiplicó por un factor superior a $20\times$ (alcanzando 756 niveles máximos para $n = 10\,000$). En aplicaciones de tiempo real o sistemas embebidos con espacio de pila acotado, este comportamiento representa un riesgo de desbordamiento de pila. En tales contextos, ShellSort, al ser un algoritmo estrictamente iterativo de memoria auxiliar $\mathcal{O}(1)$, ofrece una garantía de seguridad operativa superior (Brassard & Bratley, 1997; Musser, 1997).

5 Conclusiones

1. **Hegemonía de QuickSort en el caso general:** QuickSort con pivote central confirmó su superioridad temporal en distribuciones estocásticas y estructuradas sin repetición masiva de claves, logrando tiempos de procesamiento hasta $2.3\times$ más rápidos que ShellSort para $n = 100\,000$ elementos.

2. **Vulnerabilidad Crítica ante claves repetidas:** La variante de QuickSort con pivote central sufre una severa degradación algorítmica ante alta densidad de duplicados, elevando su complejidad operacional hacia $\mathcal{O}(n^2)$ y multiplicando por más de $20\times$ la profundidad de la pila de llamadas. En estos escenarios, se recomienda migrar a esquemas de particionamiento en tres vías o algoritmos introspectivos.

3. **Robustez y eficiencia memoria de ShellSort:** ShellSort demostró ser un algoritmo sumamente robusto, predecible y seguro. Su diseño iterativo e *in-place* previene riesgos de desbordamiento de pila y aprovecha eficientemente la localidad de referencia en la memoria caché, destacando especialmente en vectores preordenados o con alta presencia de duplicados.

4. **Criterio de selección en ingeniería de software:** Para la práctica profesional en ingeniería, la elección entre ShellSort y QuickSort no debe depender únicamente de la complejidad teórica asintótica. Es imprescindible evaluar la arquitectura de hardware, las restricciones físicas de la pila del sistema operativo y la naturaleza estocástica de los datos de entrada.

Referencias

Brassard, G., & Bratley, P. (1997). *Fundamentos de Algoritmia*. Prentice Hall.

Ciura, M. (2001). Best increments for the average case of Shellsort. *Fundamentals of Computation Theory*, *2138*, 106-117. https://doi.org/10.1007/3-540-44759-8_12

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introducción a los Algoritmos* (3rd). Cengage Learning. https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/

Frigo, M., Leiserson, C. E., Prokop, H., & Ramachandran, S. (1999). Cache-oblivious algorithms. *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (FOCS)*, 285-297. https://doi.org/10.1109/SFCS.1999.814590

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, *5*(1), 10-16. https://doi.org/10.1093/comjnl/5.1.10

Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd). Addison-Wesley. https://vivekupadhyay125.wordpress.com/wp-content/uploads/2013/08/donald-e-knuth-the-art-of-computer-programming-vol-3.pdf

Musser, D. R. (1997). Introspective sorting and selection algorithms. *Software: Practice and Experience*, *27*(8), 983-993. https://doi.org/10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#

Rodríguez, C. A., & Gómez, F. (2010). Análisis Comparativo y Empírico de Algoritmos de Ordenamiento en Estructuras Contiguas. *Revista Iberoamericana de Inteligencia Artificial*, *14*(45), 23-35. https://dialnet.unirioja.es/descarga/articulo/4527973.pdf

Sedgewick, R. (1978). Implementing Quicksort programs. *Communications of the ACM*, *21*(10), 847-857. https://doi.org/10.1145/359619.359631

Shell, D. L. (1959). A high-speed sorting procedure. *Communications of the ACM*, *2*(7), 30-32. https://doi.org/10.1145/368370.368387

Weiss, M. A. (2013). *Estructuras de Datos y Algoritmos en C++* (4th). Pearson Educación. https://www.uoitc.edu.iq/images/documents/informatics-institute/Competitive_exam/DataStructures.pdf